

Universidad Nacional de Entre Ríos

Facultad de Ingeniería

Bioingeniería

Algoritmos y Estructuras de Datos

Trabajo Práctico N° 2

Integrantes:

González, María Jimena

Riffel, Sheila Gabriela

Rodriguez, Maite

Fecha de entrega: 31 de Octubre de 2025

PROBLEMA 2. Temperaturas_DB

Kevin Kelvin es un científico que estudia el clima y, como parte de su investigación, debe consultar frecuentemente en una base de datos la temperatura del planeta tierra dentro de un rango de fechas. A su vez, el conjunto de medidas crece conforme el científico registra y agrega **mediciones** a la base de datos. Una medición está conformada por el valor de temperatura en °C (flotante) y la fecha de registro, la cual deberá ser ingresada como “dd/mm/aaaa” (string). (Internamente sugerimos emplear objetos de tipo *datetime*). Ayude a Kevin a realizar sus consultas eficientemente implementando una base de datos en memoria principal “**Temperaturas_DB**” que utilice internamente un árbol AVL.

SOLUCIÓN

Para resolver este problema se desarrolló la clase `Temperaturas_DB`, que internamente utiliza un árbol AVL como estructura de almacenamiento. El árbol AVL es un árbol binario de búsqueda balanceado.

- **Módulo `avl.py`** : Implementa el árbol AVL con operaciones de inserción, búsqueda, eliminación y recorrido en orden. Crea la clase `NodoAVL` que representa cada registro (fecha–temperatura) dentro del árbol.
- **Módulo `temperaturas.py`** : Define la clase `Temperaturas_DB`, que gestiona las mediciones de temperatura utilizando el árbol AVL.
- **Programa principal (`main.py`)**: Crea la base de datos, carga el archivo y ejecuta las pruebas y consultas.

ANÁLISIS DE COMPLEJIDAD

Método	Complejidad	Análisis
<code>guardar_temperatura(temperatura, fecha)</code>	$O(\log n)$	La inserción en un AVL requiere recorrer la altura del árbol ($O(\log n)$) y realizar, como máximo, una rotación.
<code>devolver_temperatura(fecha)</code>	$O(\log n)$	La búsqueda en un AVL requiere recorrer a lo sumo la altura del árbol, que es logarítmica.
<code>borrar_temperatura(fecha)</code>	$O(\log n)$	La eliminación busca el nodo y lo rebalancea; ambas operaciones toman $O(\log n)$.
<code>cantidad_muestras()</code>	$O(n)$	Internamente llama a <code>obtener_todos()</code> que recorre el árbol completo en orden, lo que requiere visitar los n nodos.
<code>devolver_temperaturas(fecha1, fecha2)</code>	$O(n)$	Recorre todo el árbol y luego filtra los nodos en el rango.
<code>max_temp_rango(fecha1, fecha2)</code>	$O(n)$	Recorre todos los nodos del árbol. En el peor caso, el valor máximo puede encontrarse al final del recorrido.

min_temp_rango(fecha1, fecha2)	O(n)	Recorre todos los nodos del árbol. En el peor caso, el valor mínimo puede encontrarse al final del recorrido.
temp_extremos_rango(fecha1, fecha2)	O(n)	Recorre todos los nodos del árbol para obtener las temperaturas del rango y luego calcula el mínimo y el máximo.

CONCLUSIÓN

En conclusión, al realizar el análisis de complejidad Big-O para cada método de la clase Temperaturas_DB, se observa que, si el archivo de muestras fuera mucho más extenso, el rendimiento del código no sería óptimo, ya que varios métodos requieren recorrer todos los nodos del árbol, lo que implica una complejidad $O(n)$.

En este problema, el conjunto de datos fue pequeño, por lo tanto, no se evidenciaron demoras. Sin embargo, para optimizar la implementación, podría incorporarse un contador interno en el método cantidad_muestras() para obtener una complejidad $O(1)$.

Asimismo, métodos como devolver_temperaturas() o temp_extremos_rango() podrían mejorarse evitando recorrer todo el árbol. Si se implementara una búsqueda por rango directamente sobre el árbol AVL, sería posible alcanzar una complejidad de $O(\log n + k)$, donde k representa la cantidad de elementos dentro del rango solicitado.